

Supplementary Materials to Graph Convolutional Branch and Bound

Lorenzo Sciandra^{a,b,*}, Roberto Esposito^b, Andrea Grosso^a, Laura Sacerdote^a, Cristina Zucca^a

^a*Department of Mathematics, University of Turin, Via Carlo Alberto, 10, Turin, 10123, Italy, Italy*

^b*Department of Computer Science, University of Turin, Via Pessinetto, 12, Turin, 10149, Italy, Italy*

Abstract

This article explores the integration of deep learning models into combinatorial optimization pipelines, specifically targeting NP-hard problems. Traditional exact algorithms for such problems often rely on heuristic criteria to guide the exploration of feasible solutions. In this work, we propose using neural networks to learn informative heuristics—most notably, an optimality score that estimates a solution’s proximity to the optimum. This score is used to evaluate nodes within a branch-and-bound framework, enabling a more efficient traversal of the solution space. Focusing on the Traveling Salesman Problem, we introduce Concorde, a state-of-the-art solver, and present a hybrid approach called Graph Convolutional Branch and Bound, which augments it with a graph convolutional neural network trained with a novel unsupervised training strategy that facilitates generalization to graphs of varying sizes without requiring labeled data. Empirical results demonstrate the effectiveness of the proposed method, showing a significant reduction in the number of explored branch-and-bound nodes and overall computational time. Some of the results concerning the use of the 1-tree relaxation are in the supplementary materials.

Keywords: Traveling salesman, Combinatorial optimization, Branch and bound, Graph neural network, Deep learning

1. Introduction

In these supplementary materials to the paper [Sciandra et al. \(2026\)](#), named Graph Convolutional Branch and Bound (GCBB), we analyze a classical exact solver for the Traveling Salesman Problem (TSP) and its enhanced neural version. The choice of this solver is related to its simplicity, which allows us to control each step and to recognize the effect of our hybrid version on it. As discussed in the main paper, this preliminary study aimed to explore how a Graph Neural Network

*Corresponding author

Email address: lorenzo.sciandra@unito.it (Lorenzo Sciandra)

(GNN) can be employed to improve an exact TSP solver. Implementing the code¹ from scratch provided full control over the branch-and-bound process and allowed us to identify potential areas for improving heuristic choices. Several ideas developed for the simple exact TSP solver presented here are then applied in the main paper to extend Concorde. These supplementary materials include only the essential details required to understand the experiments presented here, while all other aspects are discussed in the main paper. To help the reader, we repeat here formulas and some concepts introduced in the main paper if useful.

2. Traveling Salesman Problem and 1-tree branch and bound

Given an undirected graph $G(V, E)$ where each vertex $i \in V = \{1, 2, \dots, n\}$ represents a city and each edge $e = i, j \in E$ has an associated travel cost c_e the TSP model writes out as:

$$\min f(x) = \min \sum_{e \in E} c_e x_e \quad (1)$$

$$\text{subject to } \sum_{e \in \delta(i)} x_e = 2 \quad i \in V \setminus \{n\} \quad (2)$$

$$\sum_{e \in E} x_e = n \quad (3)$$

$$\sum_{e \in E(S)} x_e \leq |S| - 1 \quad \forall S \subset V \setminus \{1\} \quad (4)$$

$$x_e \in \{0, 1\} \quad \forall e \in E \quad (5)$$

A classical exact solver for the TSP is a 1-tree branch and bound. Precisely, following [Held and Karp \(1970\)](#) and [Held and Karp \(1971\)](#), a reasonable lower bound for the TSP is obtained on a Lagrangian relaxation on the degree constraints (2) that ensure that each vertex has exactly two incident edges except for the starting one. The solution to the relaxed problem is known as a 1-tree, and it can be efficiently computed in polynomial time using either Kruskal's or Prim's algorithm. Specifically, it involves a minimum spanning tree over all vertices except the starting one, and then adding the two edges incident to it with the lowest weights. Since in a cycle, each vertex can be the starting vertex, the one that defines the maximum lower bound and so minimizes the initial gap with the optimum is usually chosen. For this work, we implemented a branch and bound algorithm following what is proposed in [Volgenant and Jonker \(1983\)](#) and [Valenzuela and Jones \(1997\)](#): at

¹Code, data, and results are available at the following GitHub repository: <https://github.com/LorenzoSciandra/GraphConvolutionalBranchandBound>.

each node of the branch and bound tree, a fixed number of sub-gradient improvements are done to increase the value of the corresponding lower bound found with the 1-tree relaxation. If the resulting node is not closed, some edge-fixing techniques from [Benchimol et al. \(2010\)](#) are applied to further restrict, whenever possible, the number of edges that are candidates for the branching step. Only after this step, if the subproblem remains open, branching is performed using the binary branching scheme proposed in [Shutler \(2001\)](#), which integrates an edge-scoring mechanism with the subgradient optimization procedure. Specifically, when performing the subgradient optimization steps, the optimization process eventually converges to the value of the linear relaxation; in the final iterations, the relaxed solution typically oscillates among a few 1-trees. A fractional score, representing its normalized frequency of appearance in this phase, is assigned to each edge. The edge with the score closest to 0.5 is then selected for branching. A multi-start nearest-neighbor heuristic ([Flood, 1956](#)) is applied at the beginning to generate the initial feasible solution and upper bound.

3. Graph Neural Networks for the TSP

A probability matrix $\mathbf{P}$ that contains, like the one presented in the main paper, probabilities for each edge to belong to an optimal tour is here obtained through a supervisely pre-trained GNN.

More precisely, the pre-trained GNNs proposed in [Joshi et al. \(2019\)](#) are defined for graphs of fixed dimension $n = \{20, 50, 100\}$, and the architecture is implemented as follows.

At first, the input features of vertices and edges are mapped into an embedding space with dimension $h = 300$ through learnable linear transformations. Then, $L = 30$ layers of message passing with residual connections are applied, where a gating mechanism is used to filter the relevant information in the neighboring vertices through a dense attention map. In the post-processing stage, the edge embeddings $\mathbf{e}_{ij}^L$ of the last layer are fed into a Multi-Layer Perceptron to compute the probability of the corresponding edge being in an optimal tour with a sigmoid activation.

For each graph size, the training dataset consists of one million graph instances, where vertex positions are sampled uniformly from the $[0, 1] \times [0, 1]$ square, and distances are computed using the L_2 norm. During each training epoch, a subset of 10,000 instances are sampled from the training set, and the model is trained to output the symmetric probability matrix $\mathbf{P}$. So, the models were trained end-to-end in a supervised manner by minimizing the cross-entropy loss via gradient descent over these probabilities. The pre-trained models have been made publicly available by the authors of [Joshi et al. \(2019\)](#) and are used for this first set of experiments.²

²Downloadable from the paper repository <https://github.com/chaitjo/graph-convnet-tsp>

A remarkable limit of this approach lies in the reliance on supervised training, which requires an extensive dataset of optimally solved instances. This actually prevents the effective handling of large TSP instances.

4. Graph Convolutional Branch and Bound

The goal of this section is to explain how the 1-tree optimality scores O_T , formally defined in the main paper, are integrated into the 1-tree-based branch and bound. The integration takes place in different parts of the branch and bound procedure, defining the Graph Convolutional Branch and Bound.

4.1. Relaxation at the root node

The lower bound at the root node of the branching tree is computed by generating a 1-tree rooted at a suitable vertex. Among all the possible vertices, it is usually chosen the one that allows the generation of the 1-tree with the highest possible lower bound. When different 1-trees provide the same lower bound, we break such ties by choosing the one with the highest expected value $\mathbb{E}[O_T]$.

4.2. Probabilistic Nearest Neighbor

Leveraging the probabilistic information of the edges now available, we advocate utilizing an algorithm we term the Probabilistic Nearest Neighbor (PNN) for determining the initial feasible solution. The PNN starts from a specific vertex, and, at each step, chooses the edge with the maximum probability that leads to an unvisited vertex. This process is iteratively applied with all vertices as starting points. The minimum tour among all iterations is then compared with the one obtained through the classical multi-start nearest neighbor. Ultimately, only the best tour reached is regarded as the initial feasible solution.

4.3. Branching variable selection

The branching operation is performed by selecting or excluding a certain edge from the solution. We adopt the scoring scheme proposed in [Shutler \(2001\)](#) for selecting the branching edge; additionally, in this selection, we break ties by selecting the edge (i, j) with the highest p_{ij} among the edges with maximum Shutler’s score. The rationale behind this is that if the GCN assigns a high probability to an edge because it deems it likely to be part of an optimal tour, then, among peers with the same Shutler’s score, it is sensible to prioritize exploration of this edge first.

4.4. Branching node selection

In the branch and bound, when a subproblem with a value lower than the current upper bound generates its children through the branching step, these children are added to a list of open decision nodes. In the classic version of the algorithm, these nodes are ordered based on increasing values of the lower bounds given by the associated relaxations (1-trees), adhering to the standard best-first rule. In our version of the algorithm, when the difference between the lower bounds of two nodes is below a designated threshold, the node exhibiting a superior expected value of the optimality score O_T is chosen.

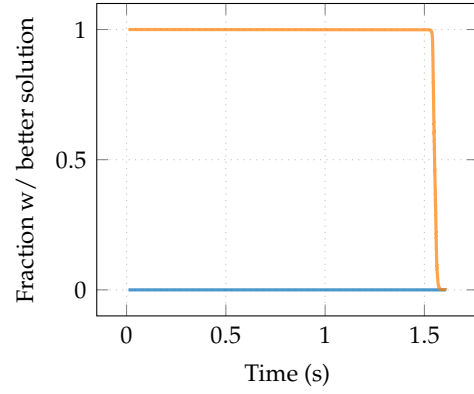
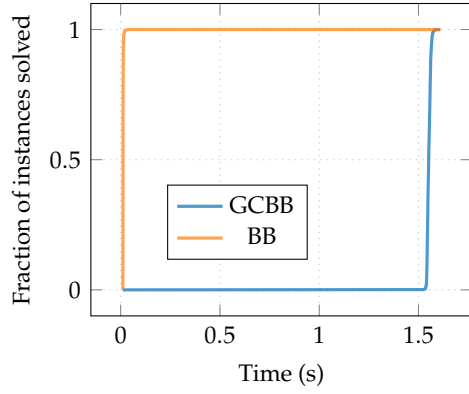
5. Results

The graph instances used for testing our Graph Convolutional Branch and Bound are randomly generated by uniformly sampling their coordinates (v_{i1}, v_{i2}) in the $[0, 1] \times [0, 1]$ square, and with Euclidean distances as defined in the TSPLIB collection. We chose to sample in this subspace because every graph can be rescaled to fit within the unit square without altering the set of optimal tours. Moreover, given that we are tackling an NP-hard problem, it is reasonable to establish a time limit, beyond which the computation is halted. When this happens, the problem instance is deemed unsolved, and its computation metrics are disregarded. In our scenario, a time limit of 10 minutes is set, serving as the exit condition. These experiments were conducted on a server that features an Intel Xeon Cascade Lake processor with 4 cores and two NVIDIA Tesla T4 GPUs, each with 15 GB of memory.

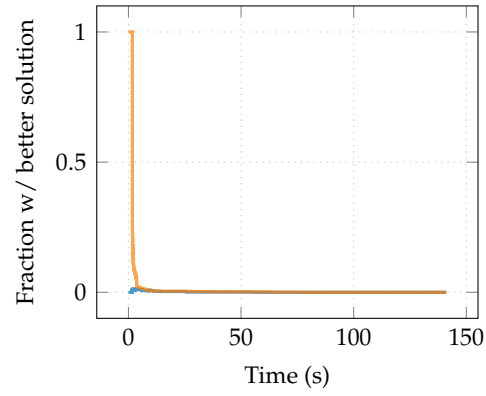
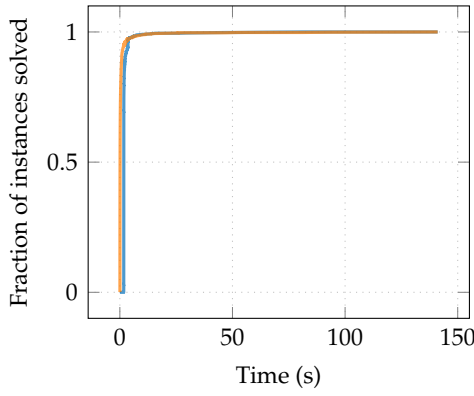
5.1. Comparison metrics

As previously said, in this first part of our work, we avoided the use of predefined libraries for standard solvers because we want to monitor various metrics about the properties of the GCBB that otherwise would be hidden by the software. In addition to conventional metrics like average solution time or average time to achieve optimality, we consider a few metrics based on the number of created nodes in the branch and bound tree, which is a measure that has the advantage of being independent of the execution environment. The combinatorial explosion of these nodes is indeed at the core of the algorithm’s high computational complexity. Specifically, we report:

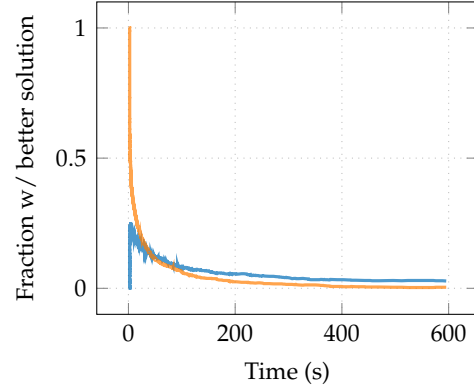
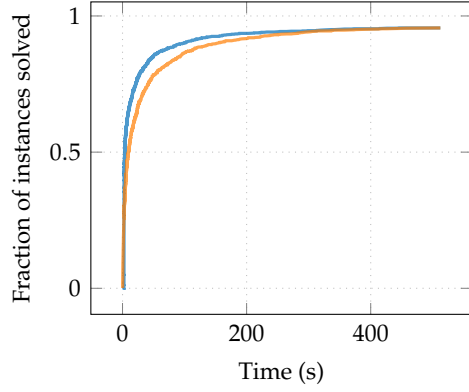
- **Total Time:** the cumulative time required for the entire procedure, encompassing instance file parsing, algorithm execution, result output, and neural network inference, if used;
- **BB Time:** the computation time dedicated exclusively to the branch-and-bound phase of the algorithm;



(a) 20 vertices.



(b) 50 vertices.



(c) 100 vertices.

Figure 1: Performances achieved by the GCBB and BB solver grouped in two distinct performance profiles for each graph size where an exact model exists.

- **Time to Best**: the elapsed time for the branch-and-bound algorithm to first discover the solution that is ultimately confirmed as optimal;
- **BB Tree Depth**: depth of the generated tree;
- **Depth of the optimum**: depth at which the optimum is found;
- **Generated BB nodes**: total number of nodes required for completing the search for the optimum;
- **Explored BB nodes**: cardinality of the subset of Generated BB nodes necessitating a branching step;
- **BB nodes before optimum**: number of nodes generated before reaching the optimum.
- **Optimality score** $\frac{\mathbb{E}[O_T]}{n}$: measures the goodness of the chosen criterion and so of the overall procedure. Indeed, we expect this value to be close to 1 for all the 1-trees that represent the optimum of the analyzed problems. See the main paper for a formal definition.

5.2. 1-tree results

We derive the value of the mentioned metrics by averaging them on the solutions obtained on 1000 instances for all graph sizes. They are presented in [Table 1](#), where optimal values are highlighted in bold. If the disparities in the means are statistically significant according to a coupled Wilcoxon signed-rank test at the 5% significance level, the values are also underlined.

As we can see, GCBB achieves better results than classic BB when the graph size grows large. Indeed, the inference of the GCN requires polynomial time in the number of vertices, while the branch and bound is exponential, and, for large instances, the disparity between the two solvers widens as the neural network overhead gets amortized. For graphs of 100 vertices, the overall hybrid procedure requires less time on average for solving an instance, while for $n = 20$ and $n = 50$ the total time is smaller for the BB approach. It is worth noting that the metrics concerning branch and bound nodes were already significantly better with instances of size greater than or equal to 50 cities. We expect this trend to emerge and even amplify with larger instances. To better analyze the differences between the two solvers, we also display performance profiles in [Figure 1](#). In each panel on the left, as time grows, we report the fraction of problem instances that are solved by each algorithm. Meanwhile, in the panel on the right, we also consider instances not yet solved. More precisely, we report the proportion of instances for which each algorithm has reached a superior objective value. We note that, in these latter plots, all lines should approach zero if the algorithms are given enough time.

Table 1: The first three rows present the percentages of the total instances solved and how the optimum is found. In the following rows, we report the mean values and their confidence intervals for all the chosen metrics. All these values are calculated when an exact GCN exists and refer to the same set of instances solved by both solvers within 10 minutes. A dash (–) is used to indicate that the corresponding metric is not defined for a given solver.

	$n = 20$		$n = 50$		$n = 100$	
	BB	GCBB	BB	GCBB	BB	GCBB
Instances solved	100.0%	100.0%	100.0%	100.0%	95.6%	95.6%
with NN	8.9%	1.3%	0.0%	0.0%	0.0%	0.0%
with PNN	–	86.0%	–	79.5%	–	48.12%
Total time (s)	<u>0.01</u> ± 0.0	1.55 ± 0.0	<u>1.65</u> ± 0.85	3.25 ± 0.77	51.69 ± 5.91	<u>45.73</u> ± 5.4
BB time (s)	0.0 ± 0.0	0.0 ± 0.0	1.62 ± 0.85	<u>1.5</u> ± 0.77	51.46 ± 5.9	<u>43.25</u> ± 5.39
Time to Best (s)	0.0 ± 0.0	0.0 ± 0.0	0.75 ± 0.36	<u>0.51</u> ± 0.24	33.60 ± 4.09	<u>19.1</u> ± 3.21
BB tree depth	<u>2.41</u> ± 0.16	2.79 ± 0.19	9.16 ± 0.39	<u>8.29</u> ± 0.37	16.41 ± 0.4	<u>15.54</u> ± 0.44
Depth of the opt.	1.61 ± 0.08	<u>0.36</u> ± 0.07	5.83 ± 0.21	<u>1.65</u> ± 0.22	11.03 ± 0.26	<u>6.44</u> ± 0.43
Generated BB nodes	<u>7.18</u> ± 1.18	7.89 ± 1.21	874.98 ± 416.66	<u>805.14</u> ± 389.44	4439.60 ± 500.27	<u>3590.77</u> ± 443.48
Explored BB nodes	<u>6.34</u> ± 1.05	7.46 ± 1.02	778.19 ± 398.87	<u>753.88</u> ± 376.0	3308.89 ± 386.92	<u>3013.26</u> ± 380.32
BB nodes before opt.	3.58 ± 0.65	<u>1.61</u> ± 0.68	388.68 ± 191.23	<u>214.31</u> ± 127.41	2347.24 ± 292.04	<u>1561.14</u> ± 273.65
$\frac{\mathbb{E}[O_T^*]}{n}$	–	0.97 ± 0.0	–	0.99 ± 0.0	–	0.99 ± 0.0

As we can observe, when the problem instance is small in size (e.g. 20), the overhead of the neural network (which requires a few seconds to run) is not justified since all the problems are solved by BB in just a few milliseconds. Instances with 50 are instead intermediate ones where the overhead is completely compensated by the quality of the extracted information, and a difference in times is not noticeable. With instances of 100, however, acquiring good heuristic information about the graph instance allows for a significant performance improvement. It’s also worth noting that for graphs of these dimensions, not all instances are solved within 10 minutes. In this situation, as can be seen from the gap between the curves in [Figure 1c](#) for 100 cities, the hybrid version terminates execution with a better solution in most cases.

Let us conclude by analyzing the average value assumed by the optimality score O_T on the optimal 1-trees. As we can see, the average score is close to 1, indicating that most of the edges predicted by the neural network as belonging to an optimal tour actually are.

6. Conclusion

This experiment prepares for the findings discussed in the main paper. In this setting, especially compared to Concorde, the information extracted by the network proves useful even for smaller

instances, since the underlying solver is less powerful.

References

- Benchimol, P., Régin, J.C., Rousseau, L.M., Rueher, M., van Hoeve, W.J., 2010. Improving the held and karp approach with constraint programming, in: Lodi, A., Milano, M., Toth, P. (Eds.), *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, Springer Berlin Heidelberg. pp. 40–44. doi:[10.1007/978-3-642-13520-0_6](https://doi.org/10.1007/978-3-642-13520-0_6).
- Flood, M.M., 1956. The traveling-salesman problem. *Operations Research* 4, 61–75. doi:[10.1287/opre.4.1.61](https://doi.org/10.1287/opre.4.1.61).
- Held, M., Karp, R.M., 1970. The traveling-salesman problem and minimum spanning trees. *Operations Research* 18, 1138–1162. doi:[10.1287/opre.18.6.1138](https://doi.org/10.1287/opre.18.6.1138).
- Held, M., Karp, R.M., 1971. The traveling-salesman problem and minimum spanning trees: Part ii. *Mathematical Programming* 1, 6–25. doi:[10.1007/BF01584070](https://doi.org/10.1007/BF01584070).
- Joshi, C.K., Laurent, T., Bresson, X., 2019. An efficient graph convolutional network technique for the travelling salesman problem. *CoRR* abs/1906.01227.
- Sciandra, L., Esposito, R., Grosso, A., Sacerdote, L., Zucca, C., 2026. Graph convolutional branch and bound. *European Journal of Operational Research* doi:doi.org/10.1016/j.ejor.2026.03.036.
- Shutler, P.M.E., 2001. An improved branching rule for the symmetric travelling salesman problem. *The Journal of the Operational Research Society* 52, 169–175. doi:[10.1057/palgrave.jors.2601053](https://doi.org/10.1057/palgrave.jors.2601053).
- Valenzuela, C.L., Jones, A.J., 1997. Estimating the held-karp lower bound for the geometric tsp. *European Journal of Operational Research* 102, 157–175. doi:[10.1016/S0377-2217\(96\)00214-7](https://doi.org/10.1016/S0377-2217(96)00214-7).
- Volgenant, T., Jonker, R., 1983. The symmetric traveling salesman problem and edge exchanges in minimal 1-trees. *European Journal of Operational Research* 12, 394–403. doi:[10.1016/0377-2217\(83\)90161-3](https://doi.org/10.1016/0377-2217(83)90161-3).